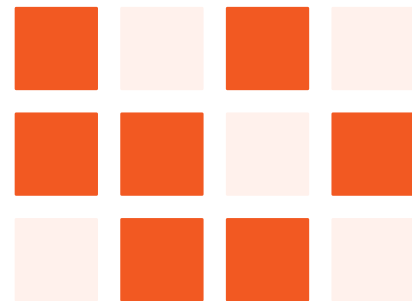


LLM 애플리케이션 프로덕션 운영, 오피저버빌리티로 풀다

신민철 · Agent Developer



PART 01

LLM, 만드는 건 쉬운데 운영은?

LLM 애플리케이션 구축

16

L I N E S

```
1 import boto3, json
2
3 client = boto3.client("bedrock-runtime")
4
5 response = client.invoke_model(
6     modelId="anthropic.claude-3-sonnet-20240229-v1:0",
7     contentType="application/json",
8     body=json.dumps({
9         "anthropic_version": "bedrock-2023-05-31",
10        "max_tokens": 1024,
11        "messages": [{"role": "user", "content": prompt}]
12    })
13 )
14
15 result = json.loads(response["body"].read())
16 print(result["content"][0]["text"])
```

01 boto3 클라이언트 생성

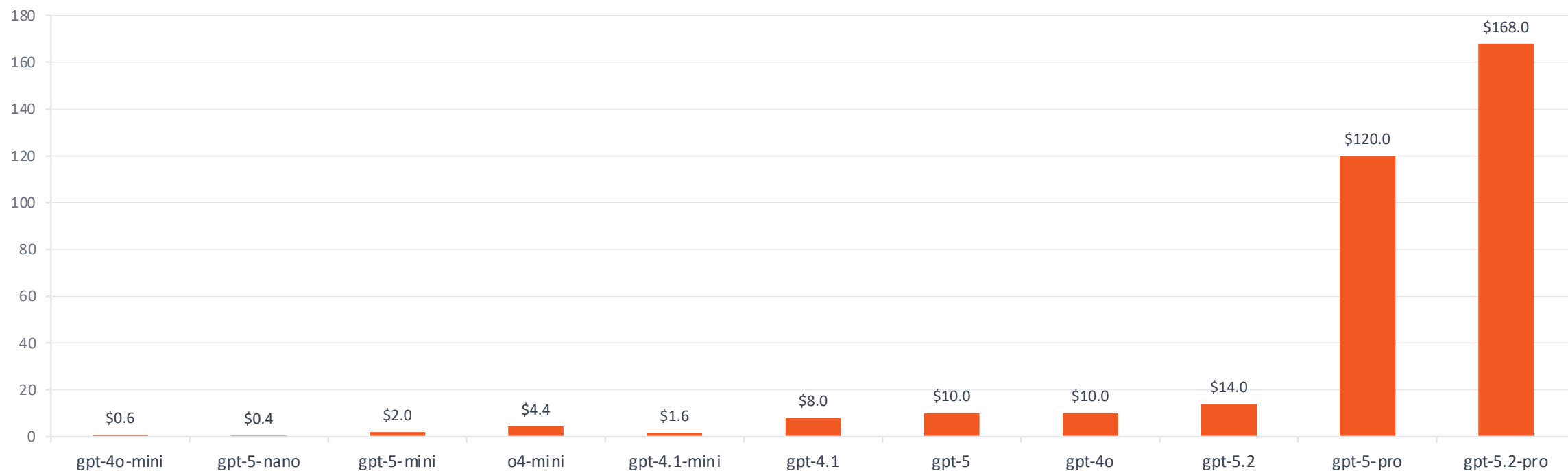
02 invoke_model 호출

03 모델 ID + 프롬프트 전달

토큰 비용 예측 불가

같은 질문, 모델만 변경 또는 Reasoning 모델 전환만으로도

비용은 **선형적으로 상승합니다.**



Air Canada 사건

AI

Air Canada Support

가족이 사망했습니다.
사별 할인 요금이 있나요?

⚠ 존재하지 않는 정책

사별 여행에 대해 할인 요금을 제공하고 있습니다.
정가 결제 후 90일 이내 환불 신청 시 할인 적용해 드립니다.

200 OK

서버는 정상 응답

CASE

챗봇이 존재하지 않는 할인 정책을 안내

법원 배상 판결

Moffatt v. Air Canada (2024)에서 법원은 챗봇의 잘못된 안내에 대한 항공사의 책임을 인정했습니다. 결과는 **200 OK**였지만, 현실은 **법적 손실**이었습니다.

할루시네이션 손실

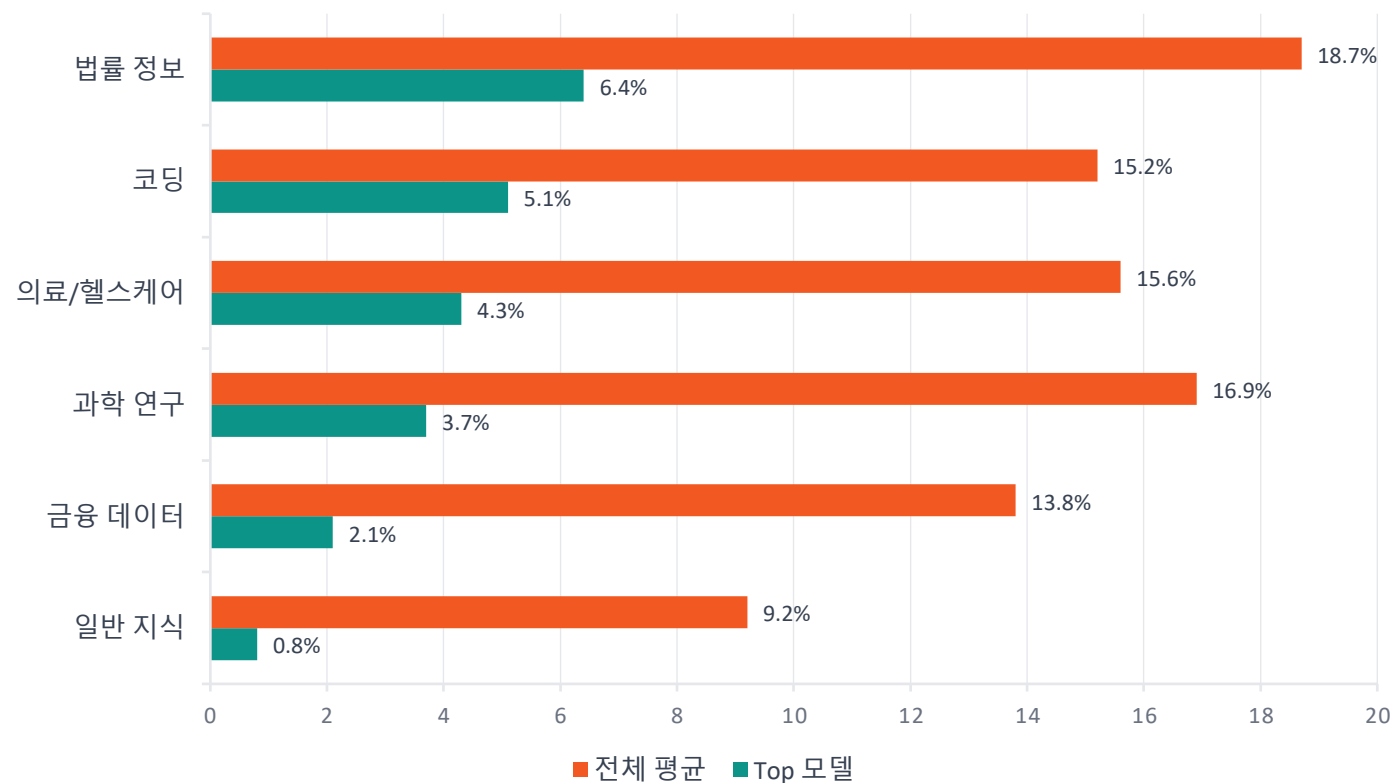
2024년 비즈니스 손실 추정치

\$67.4B

Chevy Tahoe 사건

쉐보레 챗봇이 \$1에 차량 판매에 합의 — '법적 구속력 있는 제안, 취소 불가'까지 답변.

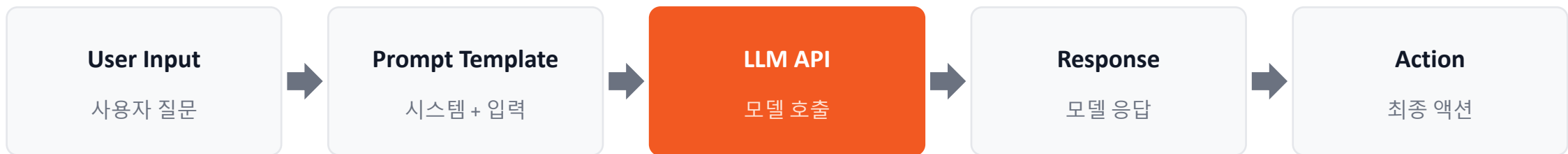
도메인별 AI 할루시네이션 발생률 (%)



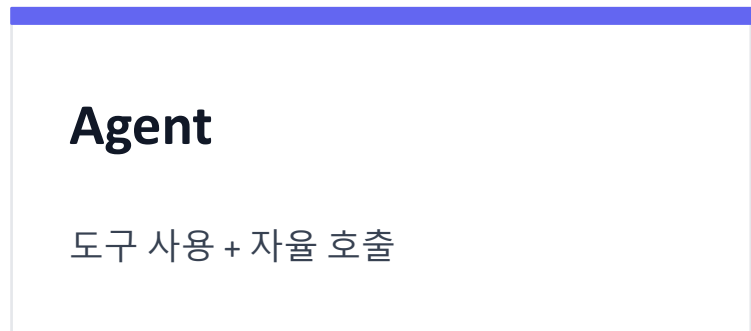
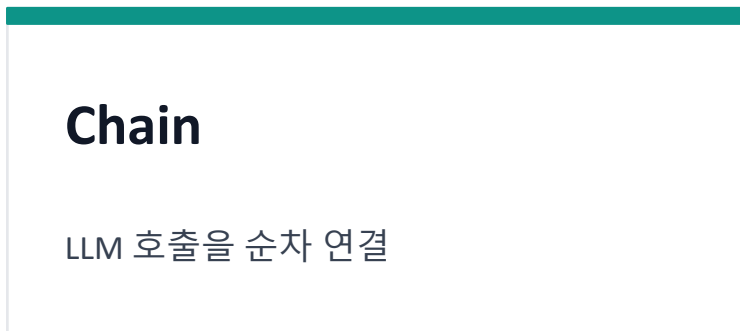
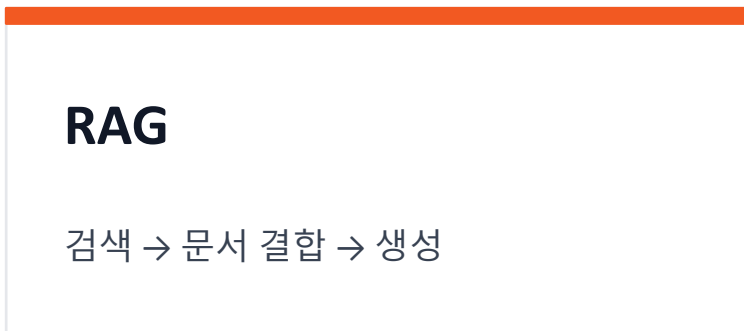
PART 02

LLM 애플리케이션이란?

LLM API 파이프라인



이 파이프라인은 패턴에 따라 구조가 달라집니다



LLM API 파이프라인의 본질적 문제

01

비결정적 출력

Non-deterministic Output

같은 입력이어도 매번 다른 답변이 나옵니다. `assert expected == actual` 패턴은 더 이상 의미가 없습니다.

`expected ≠ actual`

02

Latency 구조

Response Time Pattern

일반 API는 수십 ms 단위, LLM 호출은 수 초 단위입니다. 전체 응답 시간 예측이 어렵습니다.

`ms → seconds`

PART 03

Provider LLM vs Local LLM

두 가지 운영 모델

PROVIDER LLM

Provider LLM 사용 시

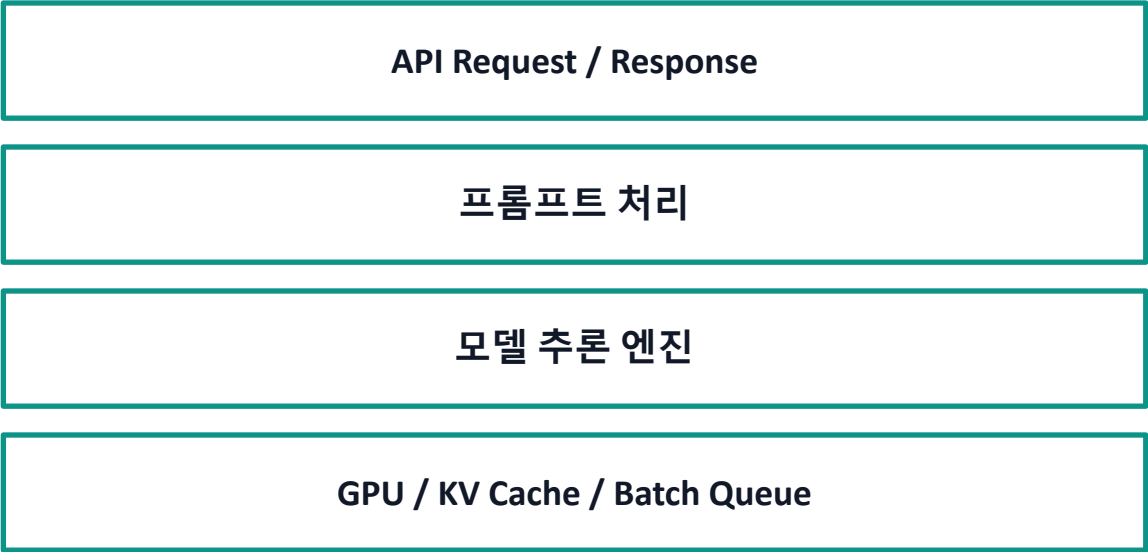
OpenAI API · Amazon Bedrock · Anthropic API



LOCAL LLM

Local LLM 서빙 시

vLLM · Ollama · TGI



Provider LLM — 모니터링 포커스

PROVIDER LLM STACK

API Request / Response

프롬프트 처리

모델 추론 엔진

GPU / KV Cache / Batch Queue

MONITORING SCOPE

API Request / Response 단일 레이어

API Latency

응답 시간

Token Count

입력 / 출력 토큰 수

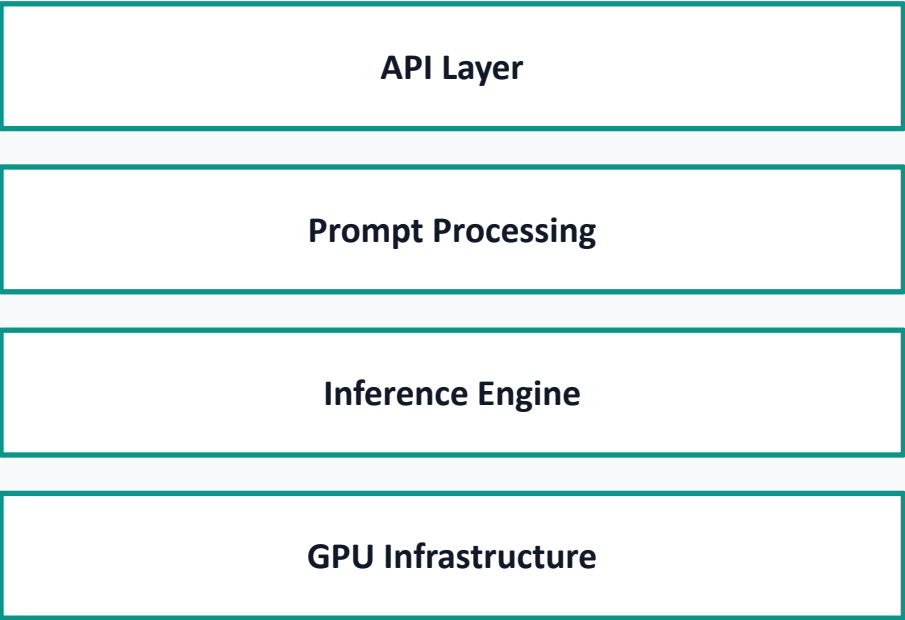
Error Rate

4xx / 5xx 에러

Local LLM — 전체 스택 모니터링

Local LLM | vLLM, Ollama, TGI (Text Generation Inference)

LOCAL LLM STACK

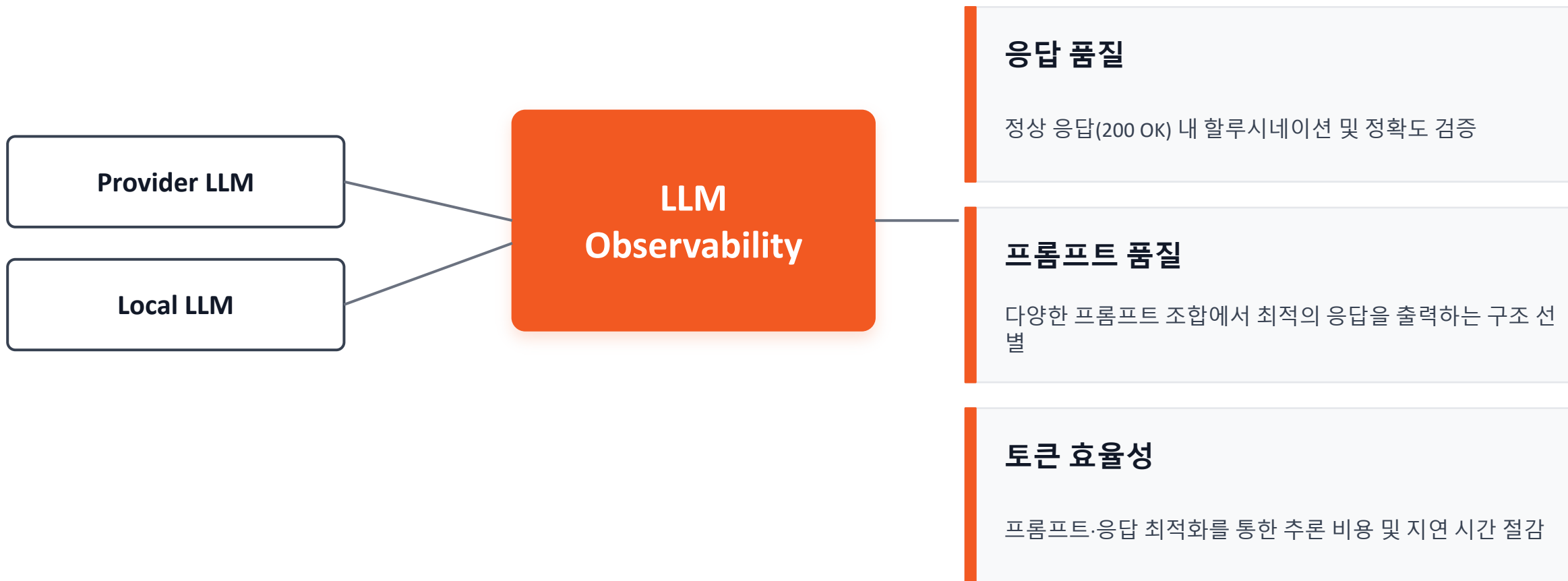


MONITORING SCOPE

4개 레이어 모두 모니터링 필요

API Latency	응답 시간
Token Count	입력 / 출력 토큰 수
TTFT, TPOT, TPS	추론 성능
VRAM, KV Cache	GPU 성능

LLM Observability — 두 모델을 모두 관통



PART 04

왜 기존 모니터링 툴로는 안되는가?

보이지 않는 두 가지 갭

GAP 01

200 OK

200 OK, 하지만 출력은 비정상

서버는 정상 응답을 반환했으나, LLM 출력 내용은 비정상.
현실은, 고객에게 잘못된 정보가 전달.

GAP 02

×10 cost

TPS 동일, 비용 10배

트랜잭션 수는 지난 달과 동일하나, LLM 비용은 10배.
현실은 APM은 트랜잭션 수는 세지만 토큰 수는 미수집.

APM이 보지 못하는 LLM 내부 구조

APM 뷰

HTTP Transaction

8,000 ms

하나의 "느린 API" 로만 보입니다

LLM Application 실제 구조

LLM 을 3번 호출한 내부 구조



■ RAG #1 · 2 s

■ LLM #2 (도구 실행) · 1 s

■ LLM #3 (종합) · 5 s

이러한 갭들을 메우는 것이 바로 **LLM Observability** 입니다.

PART 05

LLM Observability의 핵심 메트릭

3 Layers of LLM Observability

01

Application Layer

사용자 요청 단위의 전체 흐름

02

LLM API Layer

모델 호출 한 건의 성능과 비용

03

Infrastructure Layer

GPU 인프라 (Local LLM only)

Application Layer

사용자 요청 → 최종 응답까지 전체 흐름 추적

01

Trace 단위 E2E Latency

사용자 요청 한 건이 시스템 전체에서 소비한 총 시간을 분단위로 집계.

02

Chain & Agent Step

RAG, Tool 호출, 다단계 LLM 호출 등 내부 단계별 흐름과 소요 시간을 추적.

03

프롬프트와 LLM 응답

각 단계의 입력 프롬프트와 모델 응답을 모두 수집해 디버깅과 품질 평가에 활용.

LLM API Layer

모델 호출 한 건 단위로 파고드는 핵심 레이어

TTFT

Time To First Token

첫 토큰까지의 시간 — 체감 응답 속도

TPOT

Time Per Output Token

토큰 하나 생성 시간 — 스트리밍 속도 지표

TPS

Tokens Per Second

초당 처리 토큰 수 — 모델 서빙 처리량

Token Usage

Input / Output / Total

토큰 사용량 분리 추적 — 비용 원인 즉시 파악

Cost

Per-call Cost

모델별 단가 매핑 — 호출 한 건당 비용

Rate Limit

Rate Limit Hit Rate

임계치 도달 전 대응 — 트래픽 피크 관리

Infrastructure Layer

Local LLM 운영 시에만 해당되는 레이어



GPU 사용량

GPU 실제 활용률



KV Cache 사용률

추론 엔진의 핵심 지표



vRAM

GPU 메모리 사용량

Prefill vs Decode — LLM 추론의 두 단계

STAGE 01

Prefill

프롬프트를 한꺼번에 병렬 처리



↓ 병렬 Self-Attention ↓

KV-Cache 생성

STAGE 02

Decode

토큰을 한 개씩 자기회귀 생성

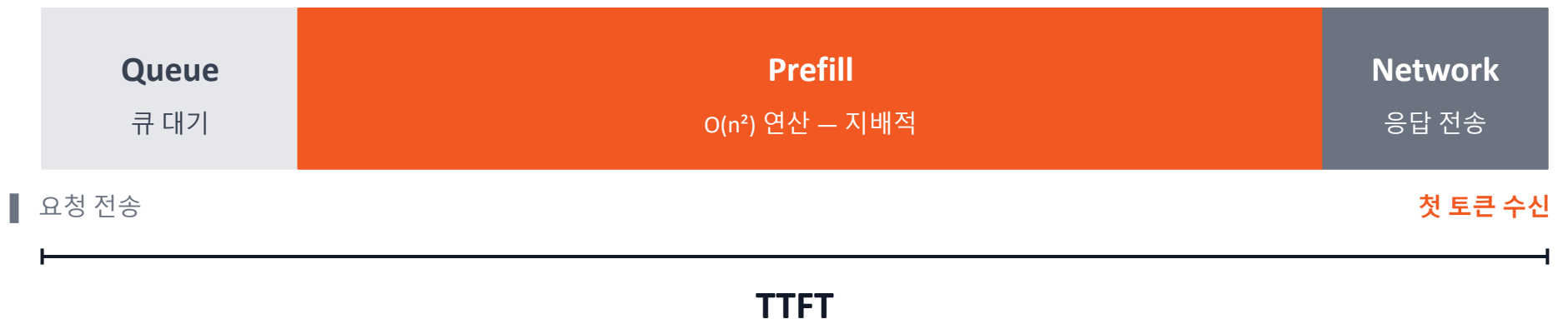


↑ KV-Cache 누적 (매 스텝) ↑



TTFT — Time to First Token

사용자가 요청을 보낸 순간부터 첫 번째 출력 토큰을 받기까지의 시간 — 체감 반응 속도 그 자체.



ITL— 토큰 사이의 간격

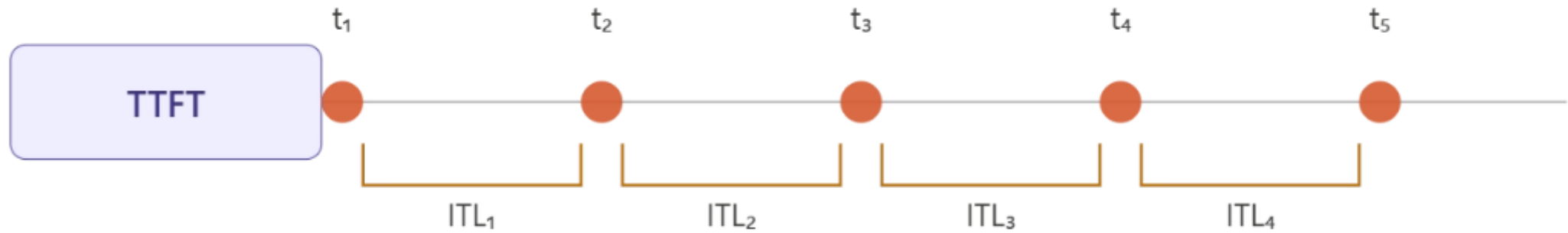
연속된 출력 토큰 사이의 평균 시간 — 스트리밍 UX에서 “글자가 얼마나 빠르게 이어서 나오는가”.



순수 디코딩 성능

$$ITL = (\text{latency} - TTFT) / (N - 1)$$

TTFT & ITL



TPS — 시스템 처리량 vs 사용자 체감



Cache Hit — KV-Cache 재사용률

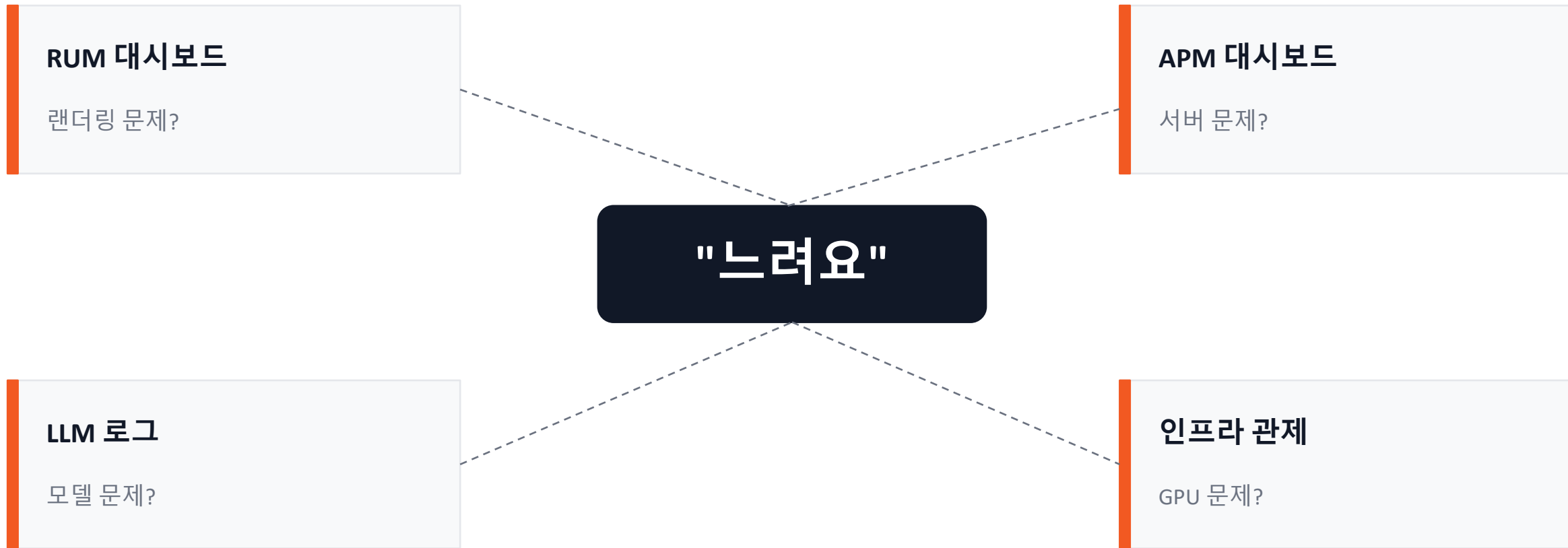
여러 요청이 동일한 접두사를 공유할 때, 그 부분의 KV-Cache 를 재계산 없이 가져오는 비율.



PART 06

통합 오피서버빌리티.

통합 오피서빌리티가 왜 필요한가요?



통합 오피서버빌리티

End-to-End Trace: 인프라와 LLM을 관통하는 단일 레이어의 완성





THANK YOU

Thank you

신민철

